

Reviewer Pack — Roe-cohomological concentration: $HX^1(X_f)$ vanishes for rand...

Entry 7c5ad65991c5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 14:43:03 UTC

Roe-cohomological concentration: $HX^1(X_f)$ vanishes for random f , separating natural-proof properties from κ

Entry ID: 7c5ad65991c5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 14:43:03 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Karchmer-Wigderson (CG-KW); coarse geometry / Roe algebras over finite KW-metric spaces **Field B** (complexity object): Boolean formula-depth lower bounds (NC^1 vs. P) — a complexity-theoretic invariant on the family of n -variate boolean functions

Statement:

Let $f : \{0,1\}^n \rightarrow \{0,1\}$ be drawn uniformly at random and let (X_f, d_f) be the KW-metric space with d_f instantiated as Hamming distance on $X_f = f^{\{-1\}}(0) \sqcup f^{\{-1\}}(1)$. For the controlled radius $R = 1$ and the discrete coarse coboundary operator δ on the Roe-style cochain complex $C^*_{\text{coarse}}(X_f)$, define $h^1(f) := \dim_R HX^1(X_f) = \dim \ker(\delta^1) - \dim \text{im}(\delta^0)$. The sub-conjecture (instance of axiom A3, anti-natural-proofs) asserts: (i) for $n \in \{3,4,5,6\}$ the empirical mean of $h^1(f)$ over uniform f tends to 0 as n grows, with $\Pr[h^1(f) > 0] \leq \exp(-\Omega(n))$; (ii) for the explicit family $f_n = \text{PARITY}_n$, $h^1(f_n) \geq n - 1$, so the trace pairing $\langle c, T \rangle$ realizes $\kappa(\text{PARITY}_n) \geq \log(R) \cdot (n-1)/n$ with $R = 2$; (iii) consequently κ is not a ‘large’ property in the Razborov-Rudich sense, since it fails the largeness criterion on the uniform ensemble while remaining nonzero on explicit functions, blocking the natural-proofs obstruction to A1.

Rationale (proposer’s reasoning):

This directly tests axiom A3 (Roe-algebraic concentration of measure). A3 predicts that κ -based proofs evade Razborov-Rudich because random f have trivial coarse cohomology — the d_f -metric of a generic f is too ‘uniformly mixed’ to support a nontrivial Roe-controlled cocycle, while structured f (PARITY, AND-OR towers) inherit nontrivial HX^1 from their algebraic stratification of $\{0,1\}^n$. If concentration fails (random f routinely have $h^1 > 0$), then A3 is wrong and the framework collides with the natural-proofs barrier; if it holds, κ is a ‘small’ invariant that still distinguishes explicit functions, which is exactly the regime needed for A1 to give a lawful $P \not\subseteq NC^1$ attack.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time:)

Framework membership: framework fw_85a254b4a0, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9c6efc5b7fa300d9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 200 uniform-random f per $n \in \{3, 4, 5, 6\}$, the empirical $\Pr[h^1(f) > 0]$ must decrease by $\geq 2\times$ per unit increase in n (monotone decay). For PARITY_ n at every $n \in \{3, 4, 5, 6\}$, $h^1(\text{PARITY}_n) \geq n-1$ AND $\log_2|(c, T)| \geq (n-1)/n$. Runtime $\leq 90s$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.82	SAFE	SAFE
NATURAL PROOFS	SAFE	0.60	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.85	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Roe algebra coarse cohomology Karchmer-Wigderson formula depth lower bound - natural proofs Razborov Rudich coarse geometry boolean function complexity - Hamming metric cochain complex parity function NC1 communication complexity

Top relevant hits considered: - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation - [<http://arxiv.org/abs/2002.07444v1>] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [<http://arxiv.org/abs/2109.13292v1>] The Hochschild Cohomology of Uniform Roe Algebras - [<http://arxiv.org/abs/0805.1385v3>] Almost-natural proofs - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/1407.6169v3>] Multiplicative Complexity of Vector Valued Boolean Functions - [<http://arxiv.org/abs/2401.17801v2>] Weighted-Hamming Metric for Parallel Channels - [<http://arxiv.org/abs/1802.02987v1>] A Generalization Method of Partitioned Activation Function for Complex Number

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import json
from collections import defaultdict

def hamming_distance(x, y):
    return sum(xi != yi for xi, yi in zip(x, y))

def build_cochain_complex(G):
    n = len(G)
```

```

E_bi = []
T_bi = []
for i in range(n):
    for j in G[i]:
        for k in G[j]:
            if hamming_distance(i, k) <= 2:
                E_bi.append((i, j, k))
                T_bi.append((i, j, k))
return E_bi, T_bi

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [3, 4, 5, 6]
    h1_counts = defaultdict(int)
    h1_positive_probabilities = []

    for n in n_values:
        X = [''.join(random.choice('01') for _ in range(n)) for _ in range(2**n)]
        G = {i: [j for j in range(len(X)) if hamming_distance(i, j) <= 1] for i in range(len(X))}

        E_bi, T_bi = build_cochain_complex(G)
        delta_0 = [[0] * len(E_bi) for _ in range(2**n)]
        delta_1 = [[0] * len(T_bi) for _ in range(len(E_bi))]

        for i in range(len(X)):
            for j in G[i]:
                if (i, j, k) in E_bi:
                    delta_0[i][E_bi.index((i, j, k))] += 1
                    delta_1[E_bi.index((i, j, k))][T_bi.index((i, j, k))] += 1

        ker_delta_1 = [sum(row) == 0 for row in delta_1]
        rank_delta_0 = sum(1 for row in delta_0 if any(x != 0 for x in row))

        h1 = sum(ker_delta_1) - rank_delta_0
        h1_counts[n] += h1
        h1_positive_probabilities.append(h1 > 0)

    empirical_mean_h1 = {n: h1_counts[n] / (2**n * 200) for n in n_values}
    fraction_positive_h1 = sum(h1_positive_probabilities) / len(n_values)

    conjecture_holds = all(empirical_mean_h1[n] <= 0.5 for n in n_values)
    counterexample = "" if conjecture_holds else "h^1(f) > 0 for some random f"

    return {
        "metric_name": "Pr[h^1(f)>0]",
        "metric_value": fraction_positive_h1,
        "instances_tested": len(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:

```

```

    result = run_trial(seed)
    print(f"TRIAL: {json.dumps(result)}")
    results.append(result)

mean_h1 = sum(result["metric_value"] for result in results) / len(results)
std_h1 = (sum((result["metric_value"] - mean_h1) ** 2 for result in results) / len(results)) ** 0.5
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_h1} std={std_h1} support_fraction={support_fraction}")
elif sum(1 for result in results if not result["conjecture_holds"]) >= len(results) * 0.2:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"h^1(f) > 0 for some random f\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cb2e6197.py", line 78, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cb2e6197.py", line 40, in run_trial
    G = {i: [j for j in range(len(X)) if hamming_distance(i, j) <= 1] for i in range(len(X))}
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cb2e6197.py", line 18, in hamming_distance
    return sum(xi != yi for xi, yi in zip(x, y))
           ~~~~~^
TypeError: 'int' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` (`'int' object is not iterable`) inside `hamming_distance` before producing any data, so no empirical results were collected for either the random ensemble or `PARITY_n`. | next: Fix the bug by representing `X` elements as bit-vectors (e.g., tuples or binary strings) rather than ints, or convert `i,j` to their binary representations inside `hamming_distance`, then rerun the trials.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	8686
2	novelty	claude_max	opus	0	0	4366
3	novelty	claude_max	opus	0	0	10951
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12827
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11079
6	judge	claude_max	opus	0	0	5790

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 53699 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/7c5ad65991c5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7c5ad65991c5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7c5ad65991c5.tar.gz` (if generated)